MICRO-PROYECTO 1-SISTEMAS DIGITALES 2: CONTRUYE UN MICROPROCESADOR RISC-V

GUÍA 1

PARA ESTUDIO INDEPENDIENTE FUERA DE CLASE

1 – Introducción a RISC-V

Revise las diapositivas "Introducción a RISC-V" y el video explicativo (https://youtu.be/Gvs7CfOmayQ). Con base en ello explique:

- Un openISA como RISC-V significa que cualquier estudiante o entusiasta pueda diseñar su propio microprocesador, montarlo en una FPGA, usar el conjunto de herramientas software creadas por terceros (como compiladores) y construir una aplicación completa que pueda vender sin pagar licencias a nadie. ¿Es esto cierto? Explique. ¿Qué ventajas tiene RISC-V?

2 – Funcionamiento básico de un microprocesador

Revise las diapositivas "Funcionamiento de un procesador" y el video explicativo (https://youtu.be/jOTJZiJgkGs). Con base en ello explique:

- El diagrama mostrado en la diapositiva 7 usa la arquitectura Neumann. ¿Qué cambios habría que hacer en el diagrama para que empleara una arquitectura Hardvard?
- Completa la siguiente tabla con una breve descripción del significado

Unidad de Control	
Contador de Programa (PC)	
ALU	
Register File	
Opcode	

3 – Especificación del RV32I

- La RV32I es la arquitectura RISC-V más básica que soporta operaciones sólo con enteros, de allí su nombre (I: Integer).
- Es una arquitectura de 32 bits, por lo que todos sus registros, incluido el contador de programa (PC), son de 32 bits.
- El register file está compuesto por 32 registros de 32 bits y reciben los nombres de x0 a x31.
- Todos los registros son de propósito general, es decir, pueden ser usados como los desee el programador. Sin embargo, algunos reciben nombres y funciones especiales con el fin de garantizar la estandarización con las herramientas software de terceros como los compiladores. En la página 47 de La Guía Práctica de RISC-V (http://www.riscvbook.com/), se encuentra el detalle de los nombres alternativos de los registros del register file.
- El registro x0 siempre se lee cero y si se escribe en este no pasa nada.
- Aunque la memoria se accede por palabras de 32 bits, es importante tener en cuenta que cada posición de memoria es de 8bits (byte), por lo cual un número de 32 bits (4 bytes) se almacena con el formato little-endian, es decir, en la dirección más baja se almacena el byte menos significativo.
- Las instrucciones soportadas por el RV32I, sus nemónicos en ensamblador y el formato de las instrucciones se muestran en la página 4 de La Guía Práctica de RISC-V (http://www.riscvbook.com/).
- En RISC-V no se especifica como se conecta el hardware externo, pues el diseñador tiene libertad de personalizar el sistema usando la estrategia de mapear los dispositivos en memoria (memory mapped I/O) lo cual se explorará más adelante en este micro-proyecto.

Preguntas:

- Según la Tarjeta de Referencia del RISC-V (Página 4 de La Guía Práctica de RISC-V, http://www.riscvbook.com/) ¿Cuáles operaciones aritméticas y lógicas puede realizar la ALU?
- Cargue el simulador en línea https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/ y revise la sección de registros y memoria. ¿Por qué en el simulador las direcciones de memoria incrementan de 4 en 4?
- Escriba y corra en el simulador el siguiente código. ¿Qué hace cada instrucción? ¿cómo se representan los números negativos en binario?

```
addi x2, x0, 100
addi x3, x0, 50
add x4, x2, x3
sub x5, x2, x3
sub x6, x3, x2
```

4 – Modos de direccionamiento e instrucciones de la ALU

Si revisa las instrucciones aritméticas de la Tarjeta de Referencia del RISC-V podrá darse cuenta de que hay dos formatos para la instrucción sumar (ADD), el formato ADD y ADD inmediato (o ADDI):

- ADD rd, rs1, rs2
  - Este formato se denomina **direccionamiento por registros** (Formato R) en el que el tanto las fuentes como el destino son registros del Register File.
  - Esta línea se debe leer como rd = rs1 + rs2.
  - Ejemplo: ADD `x3, x4, x5` ⇒ Realiza la operación x3 = x4 + x5
  - El código de máquina de esta instrucción usa el empaquetamiento R mostrado en la parte inferior de la Tarjeta de Referencia del RISC-V (ver figura de abajo).
- ADDI rd, rs1, imm
  - Este formato se denomina **direccionamiento inmediato** (Formato I) en el que una de las fuentes es un registro del Register File, mientras que el otro operando es una constante numérica que se encuentra directamente en código de máquina.
  - Esta línea se debe leer como rd = rs1 + imm
  - Ejemplo: ADD `x3, x4, 100` ⇒ Realiza la operación x3 = x4 + 100
  - El código de máquina de esta instrucción usa el empaquetamiento I mostrado en la parte inferior de la Tarjeta de Referencia del RISC-V. Nótese que, en el empaquetamiento, imm debe tomar un valor de sólo 12 bits. Al ser los registros del procesador de 32bits, todos los datos que se usen en este formato se les realiza una extensión de signo a 32 bits.

Formatos de Instrucciones de 32 bits

	31 27	26 25	24 20	19 15	14 12	11 7	6 0
R	funct7		rs2	rs1	funct3	rd	opcode
I	imm[11:0]			rs1	funct3	rd	opcode

Para entender la extensión con signo, considere el número negativo (-1). En 12 bits, este número se representa en complemento a dos como 0xFFF = 0B1111 1111 1111 (todos unos), donde el bit más significativo denota el signo. Si el número (-1) se tuviera que reescribir en 32 bits, basta con replicar el signo (bit más significativo) hasta completar 32 bits. En el ejemplo, basta con extender este bit de la forma: 0B 1111 1111 1111 1111 1111 1111 1111 1111 = 0xFFFFFFFF

Nótese que, en la tabla de instrucciones, todas las instrucciones aritméticas y lógicas soportan los modos de direccionamiento por registros e inmediato.

Preguntas:

- Con base en tabla de instrucciones de La Guía Práctica de RISC-V (pg. 44 del PDF http://www.riscvbook.com/), cómo ensamblaría manualmente a mano el siguiente programa:

```
addi x3, x0, 0x1FA      ;Nota: 0x1FA está en hexadecimal
addi x5, x0, 100        ;Nota: 100 está en decimal
sub x2, x3, x5
```

- ¿Qué hace este programa y que resultado se espera al final?

PARA EL LABORATORIO

En las próximas semanas se creará un microprocesador RISC-V multi-ciclo, es decir, en el que las instrucciones tardan varios ciclos de reloj en ejecutarse. Este tipo de microprocesador es simple de diseñar, permite reutilizar componentes, es rápido e implementable sobre FPGAs. Además, no se incluirá gestión de memoria como memoria caché, o gestión de dispositivos de entrada/salida por técnicas avanzadas como el DMA. Estos aspectos se cubrirán más adelante en el curso.

La implementación del microprocesador tendrá el aspecto mostrado en la siguiente figura, donde cada componente tiene el siguiente significado:

1. **Lógica del PC**. El PC es el registro encargado de indicar la instrucción que ejecutará la CPU. El PC siempre se incrementa por cada ciclo de reloj a menos que una instrucción de salto indique lo contrario.
2. **Unidad de Fetch (captura) y Memoria de Instrucciones (IMEM).** Es la encargada de leer los datos desde la memoria de programa (o instruction memory-IMEM). La dirección de memoria a la que se accede en la IMEM proviene del registro PC.
3. **Lógica de decodificación**. Es la parte encargada de interpretar la instrucción y activar todos los componentes necesarios para ejecutarla. Será la parte que mayor código tendremos que implementar en este mini-proyecto.
4. **Lectura del Register File**. Todas las instrucciones de un procesador RISC operan sobre los valores almacenados en una memoria interna de la CPU llamada register file. Esta es una memoria dual-port SRAM (memoria RAM estática de doble puerto), ya que en la mayoría de las instrucciones es necesario leer dos datos simultáneamente.
5. **Unidad Aritmético Lógica (Arithmetic Logic Unit-ALU).** Es el componente encargado de realizar todas las operaciones aritméticas y lógicas de la CPU, como sumas, restas, desplazamientos, etc.
6. **Escritura en el Register File.** Todas las instrucciones aritméticas y lógicas escriben los resultados de las operaciones de regreso en el register file, por lo cual es indispensable cablear la lógica para este componente.
7. **Memoria de datos (DMEM)**. Finalmente, para que una CPU sea operativa es necesario implementar las instrucciones de carga (load) y almacenamiento (store), las cuales operan sobre la memoria de datos externa.

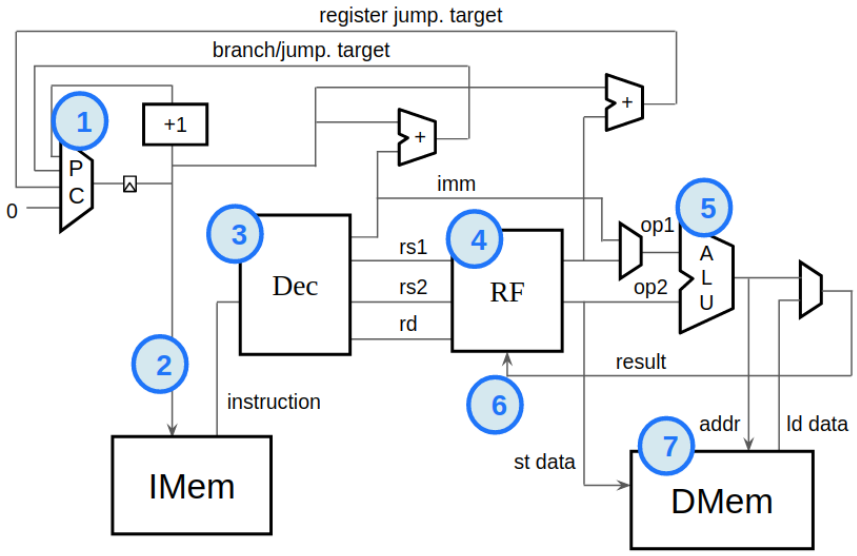


Figura 1 Diagrama de Bloques de la CPU RISC-V

En esta primera parte implementaremos dos componentes fundamentales de un microprocesador, la ALU y el registrer file (RF) usando el lenguaje de descripción de hardware Verilog. Por ello, haremos un repaso de cómo usar Verilog para especificar dos de los componentes digitales más utilizados para implementar un microprocesador: el multiplexor (MUX) y el registro de datos.

Las señales en Verilog se declaran usando la directiva `wire` o `reg`:

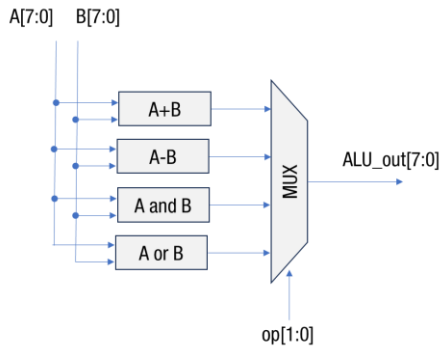
- `wire` (cable) se emplea para interconectar módulos o describir señales resultado de circuitos combinacionales. Para asignar valores a estas señales se emplea `assign`

- `reg` se emplea para almacenar valores, siempre se usan para describir circuitos secuenciales porque conducen a la generación de registros, pero también pueden servir para declarar circuitos combinacionales complejos donde no se generará un registro. Para asignar valores a se emplea `begin … end`

Un multiplexor se construye fácilmente usando el operador ternario

```
assign señal_salida = condición ? señal_si_cond_cierta: señal_si_cond_falsa
```

Vemos un ejemplo de cómo crear una ALU simple de 8 bits para las operaciones suma, resta, and y or dependiendo del valor de la señal op.




```
wire ALU_out;

assign ALU_out =
     (op == 2'b00) ? a + b:
     (op == 2'b01) ? a - b:
     (op == 2'b10) ? a & b:
     a | b;
```

Las constantes numéricas se crean indicando el número de bits del bus y el formato. A continuación, unos ejemplos:

Verilog	Descripción
`2'b01`	El número binario 01 de 2 bits
`8'd50`	El número decimal 50 codificado en 8 bits
`8'ha2`	El número hexadecimal A2 codificado en 8bits
`8'b01_000_110`	El número binario 01000110 de 8 bits (Esta opción será muy útil para construir nuestro decodificador de instrucciones).

Verilog acepta diferentes estilos de codificación. Por ejemplo, el MUX de arriba se puede describir también como:

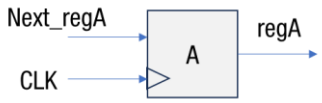
```
reg ALU_out;
always @(*) begin
     case(op)
         2'b00: ALU_out  = a+b;
         2'b01: ALU_out = a-b;
         2'b10: ALU_out =a & b;
         default: ALU_out = a | b;
     endcase
 end
```

Para extraer el rango de bits de una señal basta con escribir **`A[3:2]`** lo cual significa que de la señal A se toman solamente los bits 3 y 2.

Para concatenar varias señales se usa el formato: **`word[15:0]={upper_byte,lower_byte}`** donde word, en el ejemplo, será una señal de 16bits que se crea uniendo dos bytes (señales de 8bits). Esto será útil para realizar la extensión de signo que se mencionó en las instrucciones tipo inmediato.

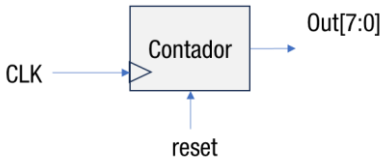
La implementación de los registros de datos en Verilog se realiza dentro de una cláusula `always @(posedge clk)` para garantizar la dependencia con la señal de reloj.

Registro de Datos




```
always @(posedge clk)
begin
    Qi <= Di;
end
```

Contador ascendente de 8 bits con reset




```
always @ (posedge clk) begin
    Q <= reset ? 0 : Q + 1;
end
```

Misión:

A) Aprendiendo a usar el simulador

- Pruebe la ALU especificada arriba. Cómo el código está incompleto, faltando la definición del módulo y archivo de pruebas. Puede apoyarse con una IA para crearlo. Por ejemplo, puede decirle "Para el siguiente código en Verilog, crea el módulo y un archivo de testbench para pruebas "…aquí pones el código verilog…".
- Si el testbench sale con directivas $display, lo recomendable es decirle a la IA que use la directiva $monitor en su lugar, lo cual produce código más simple de leer. Y si no aparece la directiva $monitor debe pedirle que la use.
- La IA generará dos códigos que debe guardar en archivos separados.
    a. El primer código es la descripción funcional del módulo, guárdelo con el nombre my_alu.v.
    b. El segundo código es el banco de pruebas o testbench, guárdelo con el nombre my_alu_tb.v (Se recomienda siempre colocar al final el nombre "_tb" para saber que se trata del banco de pruebas de my_alu).
- Para probarlos use icarusVerilog de la siguiente forma:

```
iverilog -o my_alu my_alu.v my_alu_tb.v
vvp my_alu
```

- Analice como se construye el archivo de banco de pruebas e incluya nuevas pruebas

B) Implementación del register file (RF)

Como se describió en la Sección 3, el register file es una memoria de 32 posiciones de 32bits, en la cual el elemento de la dirección 0 (registro x0), siempre se lee cero. En la siguiente figura se muestra la caja negra de un register file, donde la señal **`wr_en`** es el habilitador de escritura, es decir, si esta señal es 1, el dato que ingresa por **`wr_data`** se almacena en la posición indicada en **`wr_index.`**

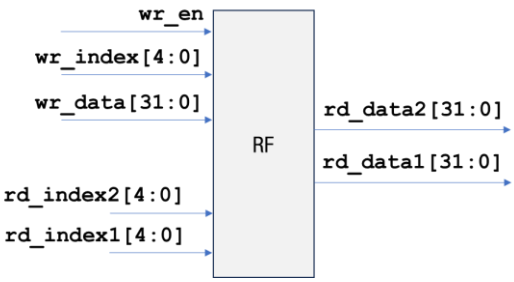


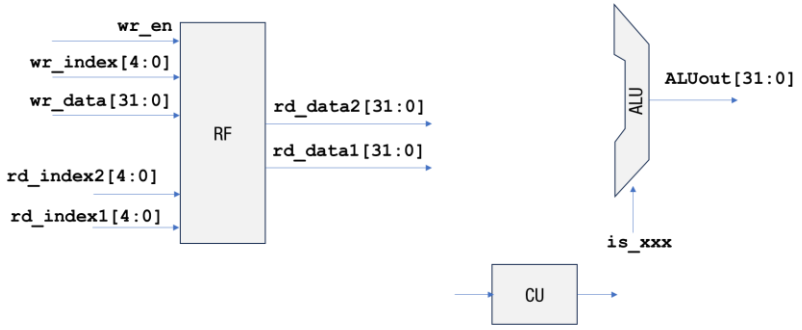
Figura 2 Register file

Por otra parte, en **`rd_data1`** saldrá el valor almacenado en la posición **`rd_index1`** de la memoria. Este tipo de memorias que cuenta con dos salidas y una entrada se denominan dual-port, cuyo diagrama se muestra a continuación (rd es la abreviación de read (lectura) y wr de write (escritura)). Por ejemplo, si se quiere leer los registros x5 y x8, **`rd_index1`** y **`rd_index2`** deben ser 5 y 8, respectivamente.

Use una IA para crear el register file dándole en el prompt la suficiente información para que se cree con la especificación dada arriba (No olvidar que x0 siempre es cero, y esto debe hacérselo saber a la IA). Debe asegurarse que el banco de registros sea sincrónico, es decir contener señal de reloj CLK, pues el microcontrolador lo probaremos en una FPGA y debemos asegurarnos de que sea sintetizable sobre estos dispositivos.

C) Implementación de las instrucciones aritmético-lógicas tipo R

Complete el cableado del siguiente diagrama para implementar solamente las instrucciones aritmético-lógicas tipo R (direccionamiento por registro) soportadas por el microprocesador RISC-V (Revise la página 4 de la guía http://www.riscvbook.com/). Para hacerlo, tenga en cuenta que por ejemplo la instrucción `add x3, x2, x1` hace la suma del contenido de los registros x2 y x1 y el resultado queda en x3, por lo tanto, ciertos campos del registro de instrucción (IR: Instruction register) se toman como entradas rd_index1, rd_index2 y wr_index. Tenga en cuenta la que unidad de control (CU: Control Unit) es quien genera las señales que controlan los diferentes módulos.



31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode

Registro de Instrucción (IR)

Una vez cableado el diagrama, transcriba el diagrama a una descripción en Verilog usando la siguiente plantilla (en rojo se advierten las partes que debe modificar):

```
1  module my_riscv (
2        input clk,              //Reloj del procesador
3        input rst,              //Reset del procesador
4        input [31:0] instr      //Instrucción de entrada
5        );
6
7    reg [31:0] IR;  //Registro de instrucción
8    always @(posedge clk) begin
9        IR <= instr;
10   end
11
12   // Señales del registro de instrucción (IR)
13   wire [6:0] funct7;
14   wire [4:0] rs2;
15   wire [4:0] rs1;
16   wire [2:0] funct3;
17   wire [4:0] rd;
18   wire [6:0] opcode;
19
20   //Extracción de los campos del registro de instrucción
```

Formatos de Instrucciones de 32 bits

	31 27	26 25 24 20	19 15	14 12	11 7	6 0
R	funct7	rs2	rs1	funct3	rd	opcode

```
21   assign funct7 = IR[31:25];
22   //... completar con el resto de extracción de campos: rs2, rs1, funct3, rd, opcode
23
24   //Salidas del register file
25   wire [31:0] src1_value;
```

```
26    wire [31:0] src2_value;
27
28    //Operandos de entrada a la alu
29    wire [31:0] alu_a;
30    wire [31:0] alu_b;
31
32    //Salida de la ALU
33    wire [31:0] alu_out;
34
35    //Señales de control de la unidad de control (CU)
36    wire wr_en;
37    wire [10:0] dec_bits;
38    wire is_add;
39    wire is_sub;
40    wire is_xxx;    //Completar con el resto de las instrucciones
41
42    //Instanciación del RF
43    RegisterFile rv_rf (
44        .clk(clk),
45        .wr_en(wr_en),
46        .wr_index( ... ), //Completar
47        .wr_data( ... ),  //Completar
48        .rd_index1( ... ),      //Completar
49        .rd_data1( src1_value ),
50        .rd_index2( ... ),      //Completar
51        .rd_data2( src2_value )
52    );
53
54    //Instanciación de la ALU
55    alu rv_alu (
56      .a( alu_a ),
57      .b( alu_b ),
58      .ALU_out( alu_out ),
59      .op({is_add,is_sub, ...})  //Completar para incluir todas las is_xxx necesarias
60    );
61
62    //Cableado entre el RF y la ALU
63    //... aquí la conexión entre srcN_value, alu_N y alu_out usando directivas assign
64
65    //Cableado de la Control Unit (UC)
66    //--------------------------------
67
68    //Señales de control
69    assign wr_en = ...;  //Completar
70    assign dec_bits = {funct7[5], funct3, opcode};
71    assign is_add = (dec_bits == 11'b0_000_0110011);
72    assign is_sub = ...; //Completar con el resto de is_xxx
73
74  endmodule
75
```

Note en el código anterior, que la instrucción almacenada en IR se parte en los campos funct7, rs2, rs1, funct3, rd, opcode (en la línea 21, usando el formato de instrucción). Estos campos de bits luego se combinan en un bus llamado dec_bits de 11 bits en la línea 70 con el fin de facilitar la decodificación de la instrucción en las líneas 71-72.

La instrucción suma se detecta en la señal is_add (línea 71) cuando el bit 5 de funct7=0, funct3=000 y opcode=0110011. Estos valores se toman de la especificación del RISC-V que se encuentra en la pág. 44 de la guía http://www.riscvbook.com/:

funct7			func3		opcode	
0000000	rs2	rs1	000	rd	0110011	R add
0100000	rs2	rs1	000	rd	0110011	R sub
0000000	rs2	rs1	001	rd	0110011	R sll
0000000	rs2	rs1	010	rd	0110011	R slt
0000000	rs2	rs1	011	rd	0110011	R sltu
0000000	rs2	rs1	100	rd	0110011	R xor
0000000	rs2	rs1	101	rd	0110011	R srl
0100000	rs2	rs1	101	rd	0110011	R sra
0000000	rs2	rs1	110	rd	0110011	R or
0000000	rs2	rs1	111	rd	0110011	R and

Debe generar las demás señales is_sub, is_and, etc., una por cada instrucción aritmético-lógica tipo R. Note también que la definición de la ALU sufrió algunos cambios. La señal op es la concatenación de las señales is_add, is_sub, etc, por lo que en este caso op tomará valores como 10000 o 01000 (un solo bit activo al tiempo) en lugar de valores binarios consecutivos. Debe modificar la ALU para incluir las instrucciones aritmético-lógicas tipo R del RISC-V y soportar estas entradas en la señal op. Finalmente, puede probar su primera aproximación de la CPU usando el siguiente testbench:

```
1  `timescale 1ns/1ps
2
3  module tb_riscv;
4
5    // Señales de prueba
6    reg  [31:0] instr;
7    reg  clk;
8    reg  rst;
9
10   // Instancia del DUT (Device Under Test)
11   my_riscv uut (
12     .clk(clk),
13     .instr(instr)
14   );
15
16   // Bloque inicial: aplica estímulos
17   integer i;
18   initial begin
19
20     // Configurar la generación de archivos de volcado
21     $dumpfile("tb_riscv.vcd");
22     $dumpvars(0, tb_riscv);
23       for(i=0; i<32; i=i+1) $dumpvars(0, uut.rv_rf.mem[i]);
24
25     //Inicia el register file para facilitar la prueba
26     //En este caso cada registro xN toma el valor de N, por ejemplo, x10 = 10.
27     for(i=0; i<32; i=i+1) uut.rv_rf.mem[i] = i;
28
29     // Muestra en consola los valores de interés cada vez que cambien
30     $monitor($time,  "  clk=%d  |  ins=%h  x4=%h  x5=%h  x6=%h",  clk,  instr,
31 uut.rv_rf.mem[4], uut.rv_rf.mem[5], uut.rv_rf.mem[6]);
32
33     #0;  clk = 0;
34              rst = 1;
35
36     #3;  rst = 0;
37        instr = 32'b0000000_00010_00011_000_00100_0110011;  //add x4, x3, x2
```

```
      $display("add x4, x3, x2");

    #12;  instr = 32'b0100000_00010_00011_000_00101_0110011;  //sub x5, x3, x2
      $display("sub x5, x3, x2");

    #12;  instr = 32'b0000000_01100_01010_111_00110_0110011;  //and x6, x10, x12
      $display("and x6, x10, x12");

     #16;
    // Fin de la simulación
    $finish;
  end

  always
    begin
        #2 clk=!clk;
    end

endmodule
```

- Incluya pruebas para las instrucciones or y xor.
- Revise en gtkwave que cada instrucción demora 3 ciclos. En el primer ciclo se carga el IR en el flanco de subida, en el siguiente flanco de subida se lee el RF y en el siguiente flanco de subida se realiza la operación (almacenamiento del resultado en el RF).